

Nicholas Carter

AI model routing, async market data, and reproducible testing

Sarasota, Florida · Open to senior/staff engineering roles and selected client work

mr.nicholas.b.carter@gmail.com · GitHub · LinkedIn

Summary

I build software that decides which AI model can handle a request, keeps market data in sync, and checks trading risk. I work in Python and TypeScript, with a focus on clear rules and failures I can reproduce.

Selected public work

Verdict · Choosing models within limits

Model routing with rules that ranking cannot override. Verdict checks capability, privacy, reliability, and policy first, then ranks only the models that qualify.

Project status: Try routing and simulated failures without API keys. The demo uses fixtures, not live providers.

Python · policy gates · receipts · failover

Prediction Market SDK · Keeping market data in sync

A typed, async Python SDK for Kalshi market data and orders. Reusable L2 order books track snapshots and deltas, while a WebSocket layer manages connections and subscriptions.

Project status: Kalshi signing, order workflows, and market-data handling are tested. Polymarket signing and live-order safeguards are still limited.

Python · AsyncIO · WebSockets · msgspec

Verdict Risk · Checking risk before action

Trading-risk checks for drawdown, position size, cluster exposure, and Kelly sizing. Pure functions keep the decisions separate from storage and network calls, so the same inputs give the same answer.

Project status: Property-based tests exercise the risk rules. This is a library of checks, not a trading system.

Python · risk gates · Hypothesis · OpenTelemetry

Capabilities

AI infrastructure

- Policy-gated model routing
- Fail-closed authorization
- Agent orchestration
- MCP integration
- Evidence receipts

Backend & data

- Python
- AsyncIO
- FastAPI
- WebSockets
- SQLite
- Typed contracts

Platform engineering

- CI/CD
- Contract parity
- Security gates
- Failure recovery
- Reproducible testing

This project-based profile covers my public software work. Follow the repository links for implementation details and current limitations.